# কোয়েরি ল্যাংগুয়েজ: RA ও SQL

**রিলেশনাল অ্যালজেবরা · SQL কমান্ড পরিবার · ছয় জয়েন · অ্যাগ্রিগেট, NULL ও GROUP BY: ১০ পাতায় পরীক্ষার চেলেঞ্জ**




↑ পুরো টপিক এক পাতায়: লাল = বেশি আসে, সবুজ = বাস্তব কাজের পাশ

## সিলেবাসের যে অংশ

ডেটাবেস সিস্টেমের সিলেবাস লেখা আছে: "Concepts of database systems; Data Models: Entity-Relationship model, Relational model; **Query Languages: Relational algebra, SQL**; Constraints and triggers; Functional dependencies and normalization; File organization and data storage; Indexing: primary and secondary indexes, B+ trees, hash tables; Query optimization; …" **এই লেসন কেবল "Query Languages" অংশটি নেয়।** কনস্ট্রেইন্ট ও ট্রিগার একই সিলেবাস লাইনের একটু নিচে — ডেটা মডেলের সাথেই দেখা হয়েছে; নরমালাইজেশন, ইনডেক্সিং ও ট্রানজ্যাকশনের নিজস্ব লেসন আছে।

### হাই-ইল্ড তালিকা

① σ বাছে **সারি**, π বাছে **কলাম**
② ρ নাম বদলায়; × প্রতিটি সারির সঙ্গে প্রতিটি সারি
③ DDL · DML · DCL · TCL — চার পরিবার
④ **DROP** কাঠামো মোছে; DELETE/TRUNCATE মোছে না
⑤ **WHERE** গ্রুপিংয়ের **আগে**, **HAVING** **পরে**
⑥ ১,NULL,৩,NULL,৫ তে **COUNT(*)=5, COUNT(a)=3**
⑦ ১০,NULL,২০ এর AVG = **১৫**, কখনো ১০ নয়
⑧ জয়েনে NULL কখনো কারো সমান হয় না
⑨ ৬ জয়েন: INNER, LEFT, RIGHT, FULL, CROSS, SELF
⑩ σ ⇒ WHERE; π ⇒ SELECT-এর তালিকা

**সংকেত:** **হলুদ** = মূল শব্দ · **লাল বৃত্ত** = অবশ্যই মুখস্থ · **বেগুনি** = মনে রাখার কৌশল · **লাল** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

## সূচিপত্র

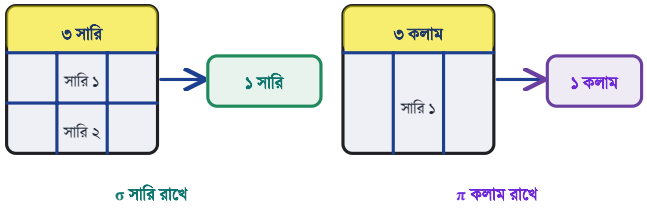

✓ রিলেশনাল অ্যালজেবরার ৮ অপারেটর … পৃ. ২
✓ Students / Courses দিয়ে RA … পৃ. ৩
✓ প্রোডাক্ট, রিনেম, জয়েন, SQL ⇄ RA … পৃ. ৪
✓ SQL: DDL, DML, DCL, TCL … পৃ. ৫
✓ অ্যাগ্রিগেট, NULL, GROUP BY … পৃ. ৬
✓ ছয় জয়েন, সারি ধরে ধরে … পৃ. ৭
✓ জয়েন প্যাটার্ন ও সাব-কোয়েরি … পৃ. ৮
✓ রাতের পরীক্ষার পাতা … পৃ. ৯
✓ অনুশীলনী MCQ + লিখিত … পৃ. ১০

# ১ রিলেশনাল অ্যালজেবরা (RA): আটটি অপারেটর

**সংজ্ঞা:** **রিলেশনাল অ্যালজেবরা** হলো কোয়েরির **তাত্ত্বিক** ভাষা: যেকোনো চাওয়ামাত্র লেখা হয় রিলেশনের উপর করা অপারেশনের একটি ধারাবাহিকতা হিসেবে, আর এক ধাপের ফলাফল পরের ধাপের ইনপুট হয়। এর **পাঁচটি মৌলিক অপারেশন** আছে (σ, π, ρ, × এবং অ্যাসাইনমেন্ট); বাকি সব — জয়েন, ইন্টারসেকশন, ডিভিশন — এগুলো থেকেই তৈরি। প্রতিটি ধাপ একটি **রিলেশন** ফেরত দেয়, তাই ধাপগুলো স্বাধীনভাবে জোড়া লাগানো যায়।

| অপারেশন | চিহ্ন | কী করে | উদাহরণ |
|---|---|---|---|
| **সিলেকশন** | σ | শর্ত পূরণ করা **সারি**গুলো রাখে | `σ salary>50000 (Emp)` |
| **প্রজেকশন** | π | বাছাই করা **কলাম** রাখে; নকল মুছে | `π name,salary (Emp)` |
| **ইউনিয়ন** | ∪ | দুই রিলেশনের সব টিপল, একবার করে | `A ∪ B` |
| **সেট ডিফারেন্স** | − | A-তে থাকা যেসব টিপল B-তে নেই | `A − B` |
| **ইন্টারসেকশন** | ∩ | দুই-তেই থাকা টিপল | `A ∩ B` |
| **কার্টেসিয়ান প্রোডাক্ট** | × | **A-র প্রতিটি সারির সঙ্গে B-র প্রতিটি সারি** | `Emp × Dept` |
| **রিনেম** | ρ | রিলেশন বা অ্যাট্রিবিউটের নাম বদলায় | `ρ pay/salary (Emp)` |
| **জয়েন** | ⋈ | প্রোডাক্ট তারপর সিলেকশন — ন্যাচারাল বা θ | `Emp ⋈ Dept` |

**ডেরাইভড অপারেশন:** মৌলিক নয় এমন সবই এই পাঁচটি দিয়ে তৈরি: **θ-জয়েন** = $\sigma_{A=B}(R \times S)$ · **ন্যাচারাল জয়েন** = θ-জয়েনের সঙ্গে জয়েন অ্যাট্রিবিউটের নকল কপি সরানো · **ইন্টারসেকশন** = `A − (A − B)` · **রিনেম** মৌলিক, কিন্তু সেলফ-জয়েন বৈধ করে তো এটাই।




কোয়েরি একটি ধারা: এক রিলেশন পরেরটিকে খায়

ভেতর থেকে বাইরে পড়ো

σ নামানে নামায়, π পাশে পাশে কাটে

**মনে রাখার কৌশল:** **"সিগমা সারি বাছে, পাই কলাম বাছে।"** σ হলো সেই অক্ষর যা দিয়ে টেবিলের একটি *লাইন* বোঝানো হয়; π ব্যবহার হয় একটি *তালিকা* বোঝাতে। SQL-এ এরা হলো **WHERE** ও **SELECT** — কখনোই অদল-বদল নয়।

**পরীক্ষার ফাঁদ:** **σ কলামের সংখ্যা বদলায় না** আর **π সারির সংখ্যা বদলায় না** (এটি নকল সরাতে পারে)। "কোন অপারেটর কলাম সরায়?" → **π**। "কোন অপারেটর সারি সরায়?" → **σ**। আর **ρ কিছুই বদলায় না**: শুধু নাম।

**সমাধানকৃত উদাহরণ ✍️ সেট অপারেটরগুলো:** ধরি `A = {1, 2, 3}` আর `B = {3, 4}`। তাহলে **A ∪ B = {1,2,3,4}** (নকল ৩ একবারই), **A − B = {1,2}**, **A ∩ B = {3}**, আর **A × B = ৬ টিপল** (৩ সারি × ২ সারি) — প্রোডাক্টে একটিও ডুপ্লিকেট থাকে না, প্রতিটি জোড়া আলাদা।

**পরীক্ষার ফাঁদ:** ইউনিয়ন, ইন্টারসেকশন ও ডিফারেন্স কাজ করে শুধু **ইউনিয়ন-কমপ্যাটিবল** রিলেশনে: অ্যাট্রিবিউটের সংখ্যা, ক্রম ও ডোমেইন এক হতে হয়। আর **π আর σ-এর ফলাফলে নকল থাকে না**, কারণ RA সেটের উপর কাজ করে — SQL-এর ব্যাগের উপর নয়।

# ২ Students / Courses ডেটাবেসে RA

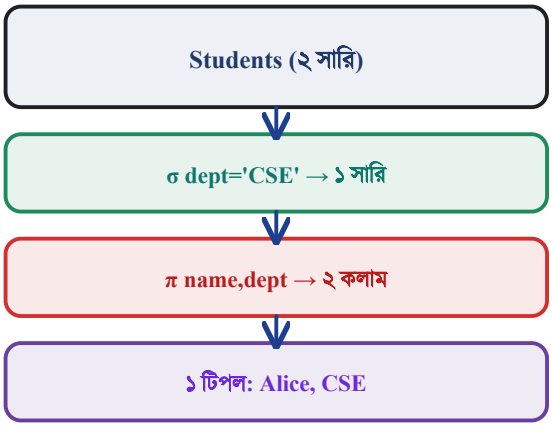
পরীক্ষায় আসা প্রতিটি RA প্রশ্ন কোনো না কোনো নির্দিষ্ট জোড়া রিলেশন নিয়ে করা হয়। এই দুটি মুখস্থ করলে চিহ্নটি আর বিমূর্ত হয় না।

| Students | id | name | dept |
|---|---|---|---|
| | 1 | Alice | CSE |
| | 2 | Bob | EEE |
| **Courses** | **cid** | **name** | |
| | C1 | DBMS | |
| | C2 | Algorithms | |

| RA কোয়েরি | ফলাফল |
|---|---|
| σ dept='CSE'(Students) | ১ টিপল: (1,Alice,CSE) |
| π name (Students) | ২ টিপল: Alice, Bob |
| π dept (Students) | ২ টিপল: CSE, EEE |
| Students × Courses | ২ × ২ = **৪** টিপল |

**পরীক্ষার ফাঁদ:** এই দুটি রিলেশনের কার্টেসিয়ান প্রোডাক্ট হলো **৪** টিপল — ২ নয়, ৩-ও নয়: **সারি** × **সারি**। সাধারণত $m$ সারির রিলেশন × $n$ সারির রিলেশন = $m \times n$ সারি — ৩ সারির দুটি টেবিলে **৯**।

**সমাধানকৃত উদাহরণ ১ ✍️:** "সব CSE ছাত্রের নাম ও বিভাগ দেখাও।"
**RA:** π name,dept (σ dept='CSE'(Students))
**SQL:** `SELECT name, dept FROM Students WHERE dept='CSE';`
RA **ভেতর থেকে বাইরে** পড়ো: আগের বন্ধনীটি আগে চলে, তাই σ ছাঁকে আর π পরে কলাম কাটে।



ভেতর থেকে বাইরে গণনা

**সমাধানকৃত উদাহরণ ২ ✍️:** "প্রতিটি ছাত্রকে প্রতিটি কোর্সের সঙ্গে জোড়া করে দেখাও।"
**RA:** π name,cname (Students × ρ cname/name(Courses))
**SQL:** `SELECT s.name, c.name FROM Students s, Courses c;`
ρ এখানে **আবশ্যক**: প্রোডাক্টের পর দুই রিলেশনেই *name* নামের অ্যাট্রিবিউট থাকে, আর একটি রিলেশনে একই নামের দুটি অ্যাট্রিবিউট থাকতে পারে না।

**সমাধানকৃত উদাহরণ ৩ ✍️:** "CSE ছাত্রদের নাম একবার করে দেখাও।"
→ **RA:** π name (σ dept='CSE'(Students)) → **Alice**-ই। σ আগে চলে, তাই দ্বিতীয় ছাত্র π গোনার আগেই বাদ পড়েছে।

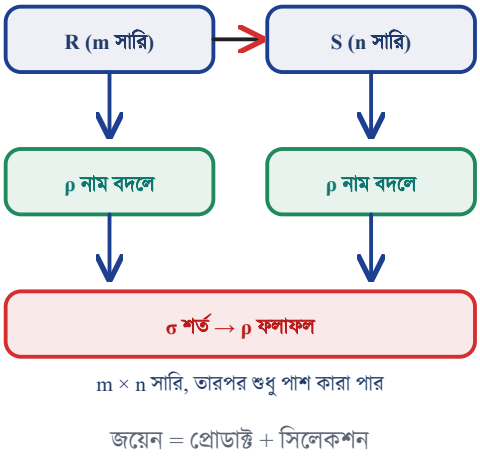
**পরীক্ষার ফাঁদ ✍️ বিখ্যাত জোড়া:** কোনো প্রশ্নে σ name (Students) ও π dept (Students) — এই দুটি দেওয়া হয়। প্রথমটি **টাইপ এরর**: *name* একটি অ্যাট্রিবিউট, শর্ত নয়, তাই σ-এর একটি **শর্ত** লাগে আর π-এর একটি **অ্যাট্রিবিউট তালিকা** লাগে। আর "π নকল সরায়" — এটি **RA**-র সত্য; সাধারণ SQL-এর `SELECT` নকল রাখে, `DISTINCT` না লিখলে।

# ৩ প্রোডাক্ট, রিনেম ও জয়েন — আর SQL-এ অনুবাদ

**জয়েনের সূত্র:** **θ-জয়েন** $R \bowtie_{A=\theta B} S \equiv \sigma_{A=\theta B}(R \times S)$ · **ন্যাচারাল জয়েন** $R \bowtie S$ যেসব অ্যাট্রিবিউট দুই রিলেশনে **একই নামে** আছে তার উপর জোড়া লাগায়, তারপর নকল কপিটি সরায়।

**ρ কেন দরকার:** একটি রিলেশনে একই নামের দুটি অ্যাট্রিবিউট থাকতে পারে না, তাই এক কপি রিনেম না করে `Emp × Emp` অবৈধ — এ কারণেই **প্রতিটি সেলফ-জয়েনে দুটি অ্যালিয়াস লাগে**।

**মনে রাখার কৌশল:** **"আগে প্রোডাক্ট, তারপর বাছাই।"** এই এক বাক্য থেকেই জয়েন, সেমি-জয়েন আর প্রশ্নে আসা প্রতিটি θ-জয়েন পাওয়া যায়। একটি WHERE-ক্লজযুক্ত কোয়েরি *দেখতে* লাগে "প্রথমে জয়েন, তারপর ছাঁকা" — অপটিমাইজার সেটি নিঃশব্দে এই আকারেই সাজিয়ে নেয়।




| ইংরেজি প্রশ্ন | রিলেশনাল অ্যালজেবরা | SQL |
|---|---|---|
| ৫০ হাজারের বেশি বেতনভোগী | `σ salary>50000 (Emp)` | `SELECT * FROM Emp WHERE salary > 50000` |
| শুধু নাম ও বেতন | `π name,salary (Emp)` | `SELECT name, salary FROM Emp` |
| সব CSE ছাত্র | `σ dept='CSE'(Students)` | `… WHERE dept='CSE'` |
| বেতনের নাম বদলে pay | `ρ pay/salary (Emp)` | `SELECT salary AS pay` |
| CSE বিভাগের ছাত্ররা | `Students ⋈ Dept` | `… JOIN Dept ON …` |

| বৈশিষ্ট্য | রিলেশনাল অ্যালজেবরা | SQL |
|---|---|---|
| প্রকৃতি | **তাত্ত্বিক** ভিত্তি | **বাস্তব**, ব্যবহারকারী ও অ্যাপ লেখে |
| ধরন | প্রসিডিউরাল — *কীভাবে* বলে | **ডিক্লারেটিভ** — শুধু *কী* বলে |
| নকল | থাকে না (সেট থিওরি); π সরিয়ে দেয় | `DISTINCT` না লিখলে থাকে |
| "৫০ হাজারের বেশি" | `σ salary>50000 (Emp)` | `SELECT * FROM Emp WHERE salary>50000` |
| অপটিমাইজেশন | পুনর্লিখনে ঠিক সম রিলেশন পাওয়া যায় | অপটিমাইজার **SQL-কে RA-ট্রিতে রূপান্তর** করে সবচেয়ে সস্তাটি বেছে নেয় |

**কেন এটি গুরুত্বপূর্ণ:** ব্যবহারকারী SQL লেখেন, আর ডেটাবেসের **কোয়েরি অপটিমাইজার** সেটিকে একটি **অ্যালজেব্রা ট্রি**তে নতুন করে লেখে, তারপর সবচেয়ে সস্তা সমতুল্য ট্রিটি বেছে নেয় — সিলেকশনগুলো নিচের দিকে ঠেলে দিয়ে যাতে বিশাল প্রোডাক্ট কখনোই না হয়। RA-এর পুরো উদ্দেশ্যই এটাই। কোন ইনডেক্স কোন ট্রিকে সস্তা করে, সেটা ইনডেক্সিং লেসনের বিষয় (**db4**)।

**পরীক্ষার ফাঁদ:** RA কাজ করে **সেট** উপর, SQL কাজ করে **ব্যাগ** উপর — তাই একই প্রশ্নে দুই ভাষা ভিন্ন সারির সংখ্যা দিতে পারে। এটা ইচ্ছাকৃত ও পরীক্ষাযোগ্য পার্থক্য, কোনো বাগ নয়। আর SQL ডিক্লারেটিভ বলে অপারেশনের ক্রম ব্যবহারকারী ঠিক করে না — অপটিমাইজার করে, সিলেকশনগুলো নিচের দিকে ঠেলে দিয়ে যাতে বিশাল প্রোডাক্ট কখনোই না হয়।

## ৪ SQL কমান্ডের প্রকার ও ক্লজগুলোর চলার ক্রম

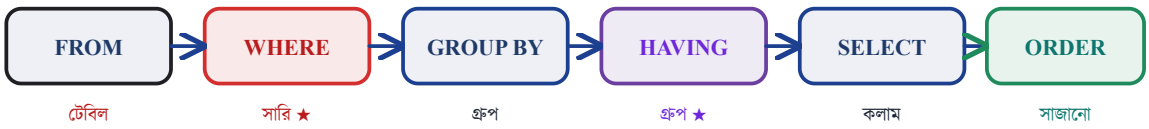
| পরিবার | পূর্ণ নাম | কমান্ড | যার উপর কাজ করে |
|---|---|---|---|
| **DDL** | ডেটা ডেফিনিশন ল্যাংগুয়েজ | CREATE, ALTER, DROP, TRUNCATE, **RENAME** | **কাঠামো** |
| **DML** | ডেটা ম্যানিপুলেশন ল্যাংগুয়েজ | SELECT, INSERT, UPDATE, DELETE (আরও MERGE) | **সারি** |
| **DCL** | ডেটা কন্ট্রোল ল্যাংগুয়েজ | GRANT, REVOKE | **অধিকার** |
| **TCL** | ট্রানজ্যাকশন কন্ট্রোল ল্যাংগুয়েজ | COMMIT, ROLLBACK, SAVEPOINT | **ট্রানজ্যাকশন** |

> D D M C T

| কমান্ড | ডেটা | কাঠামো | WHERE চলবে? | পরিবার |
|---|---|---|---|---|
| **DROP TABLE** | **সব সারি মোছে** | **মোছে** | না | DDL |
| **DELETE** | মিলে যাওয়া সারি মোছে | **থাকে** | **হ্যাঁ** | DML |
| **TRUNCATE** | সব সারি মোছে | **থাকে** | না | DDL |
| **ALTER** | থাকে | **বদলায়** | না | DDL |

```
SELECT   name, salary          -- 5   কলাম বেছে নেওয়া
FROM     Employees             -- 1   টেবিল থেকে শুরু
WHERE    salary > 50000        -- 2   সারি ছেঁকে নেওয়া
GROUP BY dept_id               -- 3   গ্রুপ বানানো
HAVING   COUNT(*) > 3          -- 4   গ্রুপ ছেঁকে নেওয়া
ORDER BY salary DESC;          -- 6   সাজানো

INSERT INTO Students (id, name, dept) VALUES (1, 'Alice', 'CSE');
UPDATE Employees SET salary = salary * 1.10 WHERE dept = 'IT';
DELETE FROM Students WHERE id = 1;        -- WHERE দিলে DELETE নির্বাচনী হয়
```




ক্লজগুলো *চলে* যে ক্রমে — আপনি যে ক্রমে লেখেন তা নয়

**পরীক্ষার ফাঁদ:** **WHERE সারি ছাঁকে গ্রুপিংয়ের *আগে*; HAVING গ্রুপ ছাঁকে গ্রুপিংয়ের *পরে*।** পুরো পার্থক্য এটুকুই, আর অ্যাগ্রিগেট ব্যবহার করতে পারে শুধু HAVING — যেমন `HAVING COUNT(*) > 5`।

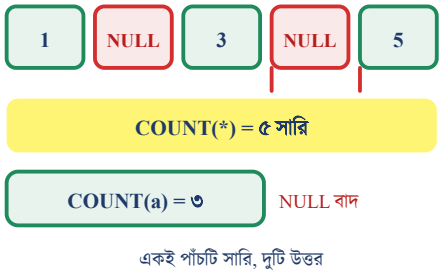
**মনে রাখার কৌশল ও অতিরিক্ত তথ্য:** **"যেখানে আগে, হেভিং পরে।"** *W*here আসে *W*hen সারি এখনো সারি। `GROUP BY` ছাড়াই `HAVING` বৈধ: পুরো টেবিলটাই একটি গ্রুপ ধরা হয়, তাই `SELECT COUNT(*) FROM Emp HAVING COUNT(*) > 5` হয় একটি সংখ্যা, নয়তে কিছু নয়।

# ৫ অ্যাগ্রিগেট ফাংশন ও NULL-এর নিয়ম

| অ্যাগ্রিগেট | যা ফেরত দেয় | কলামে NULL থাকলে | খালি সেট |
|---|---|---|---|
| **COUNT(*)** | **সারি**র সংখ্যা | **তবুও গোনে** | 0 |
| **COUNT(col)** | col-এর non-NULL মানের সংখ্যা | বাদ দেয় | 0 |
| **SUM / AVG(col)** | non-NULL মানের যোগফল / গড় | **বাদ দেয় — ০ ধরে না** | NULL |
| **MIN / MAX(col)** | সবচেয়ে ছোট / বড় non-NULL মান | বাদ দেয় | NULL |

> NULL দেখে শুধু COUNT(*)

**সমাধানকৃত উদাহরণ ১ ✍️ COUNT:** `T(a)`-তে আছে `1, NULL, 3, NULL, 5`।
`SELECT COUNT(*), COUNT(a) FROM T;` → **5, 3**
**COUNT(*)** গোনে *সারি*, তাই দুটো NULL সারিও গোনা হয়; **COUNT(a)** গোনে *a-এর মান*, আর NULL একটি মান নয়।



একই পাঁচটি সারি, দুটি উত্তর

একই পাঁচটি সারি, দুটি গণনা

**সমাধানকৃত উদাহরণ ২ ✍️ AVG:** `R(score)`-তে আছে `10, NULL, 20`।
`SELECT AVG(score) FROM R;` → (10 + 20) / 2 = **15**
`COUNT(*)` ছাড়া প্রতিটি অ্যাগ্রিগেট NULL উপেক্ষা করে; NULL-কে 0 ধরলে (10 + 0 + 20) / 3 = 10 হতো — এটিই **ভুল** উত্তর।

**সমাধানকৃত উদাহরণ ৩ ✍️ GROUP BY + HAVING:**
`Sales(region, amount)` = ('N',100), ('N',200), ('S',50), ('S',70), ('E',300)
`SELECT region, SUM(amount) FROM Sales GROUP BY region HAVING SUM(amount) > 150;`
যোগফল: N = 300, S = 120, E = 300। 150 পেরোনো শুধু N ও E → **২ সারি**।

**পরীক্ষার ফাঁদ ✍️ তিনটি:** **১.** `10, NULL, 20`-এর AVG হলো **15**, 10 নয় — NULL হলো **অজানা**, শূন্য নয়। **২.** খালি টেবিলে `COUNT(*)` = 0, কিন্তু AVG, SUM, MIN, MAX ফেরত দেয় NULL। **৩.** NULL-এর উপর যেকোনো শর্ত **অজানা, কখনো সত্য নয়**, তাই `WHERE a > 0` নিঃশব্দে NULL সারিগুলো বাদ দেয়।

**মনে রাখার কৌশল:** **"তারকা প্রতিটি সারিই দেখে, কলাম শুধু আসল উত্তর দেখে।"** তারকা পুরো টিপলের পরিচয় দেয়, তাই মান অনুপস্থিত থাকলেও সারি গোনা হয়। `COUNT(*)` পড়ো "সারি গোনো", `COUNT(col)` পড়ো "উত্তর গোনো"। জয়েনের সঙ্গে `COUNT(*)` ব্যবহার করলে সাব-কোয়েরির প্রতিটি মিল-পঙ্‌ক্তি একবার গোনা হয়, তাই জয়েনের আগে গোনা হয় না।

# ৬ ছয়টি SQL জয়েন, সারি ধরে ধরে

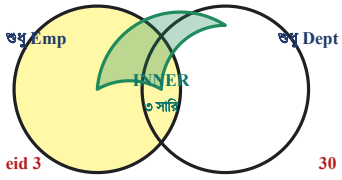
**Emp(eid, dept_id)**

| eid | dept_id |
|---|---|
| 1 | 10 |
| 2 | 20 |
| 3 | **NULL** |
| 4 | 10 |

**Dept(dept_id, name)**

| dept_id | name |
|---|---|
| 10 | IT |
| 20 | HR |
| 30 | Finance |

| জয়েন | কী রাখে | সারি |
|---|---|---|
| **INNER** | শুধু মিল | **3** |
| **LEFT** | সব Emp | **4** |
| **RIGHT** | সব Dept | **4** |
| **FULL OUTER** | সব দুটো | **5** |
| **CROSS** | সব জোড়া | **12** |
| **SELF** | নিজের সঙ্গে | সারি-জোড়া |




FULL OUTER = 3 + 1 + 1 = 5

তিনটি অংশ ও সারির সংখ্যা

**INNER JOIN ✍️:** `SELECT e.eid, d.name FROM Emp e JOIN Dept d ON e.dept_id = d.dept_id;`
eid 1 → 10 IT · eid 2 → 20 HR · eid 4 → 10 IT। eid 3-এর মান NULL, আর **NULL = 10 সত্য নয়**, তাই সে বাদ পড়ে; dept 30-এর কোনো কর্মচারী নেই। → **৩ সারি**

**LEFT ও RIGHT JOIN ✍️:** **LEFT** = ৩টি মিল + মিলহীন কর্মচারী (NULL দিয়ে পূরণ) → **৪ সারি**: (1,IT) (2,HR) (3,NULL) (4,IT)। **RIGHT** = ৩টি মিল + মিলহীন বিভাগ (NULL দিয়ে পূরণ) → **৪ সারি**: (1,IT) (2,HR) (4,IT) (NULL,Finance)।

**FULL OUTER ও CROSS JOIN ✍️:** FULL OUTER = ৩টি মিল জোড়া + ১টি মিলহীন *Emp* সারি (eid 3) + ১টি মিলহীন *Dept* সারি (30 Finance) = 3 + 1 + 1 = **৫ সারি**; সূত্র **INNER + শুধু-LEFT + শুধু-RIGHT**। CROSS JOIN হলো ৪ সারি × ৩ সারি = **১২ সারি**, কোনো শর্ত ছাড়াই প্রতিটি সারির প্রতিটি সারির সঙ্গে; সাধারণত $m \times n$, তাই ৩ সারির দুটি টেবিলে **৯**।

**SELF JOIN:** একটি রিলেশনকে **দুটি অ্যালিয়াস** নিয়ে নিজের সঙ্গে জোড়া লাগানো, যাতে একই টেবিলের সারিগুলোর সম্পর্ক দেখানো যায় — কর্মচারী তার ম্যানেজারের সঙ্গে। *Emp* টেবিলে আরও দুটি কলাম যোগ করুন — `name` এবং `mgr_id` (ম্যানেজারের `eid`) — তারপর:
`SELECT a.name AS emp, b.name AS mgr FROM Emp a JOIN Emp b ON a.mgr_id = b.eid;`
দুটি অ্যালিয়াস **আবশ্যক**: না দিলে শর্তটি একটি কলামকে নিজের সঙ্গে তুলনা করত।

**পরীক্ষার ফাঁদ ✍️ NULL-এর ফাঁদ:** যে কর্মচারীর `dept_id` NULL, সে কোনো **inner-join** বা **right-join** ফলাফলে থাকে না; **LEFT** ও **FULL OUTER** জয়েনে সে থাকে, NULL দিয়ে পূরণ করে। "আমার সারি কোথায় গেল?" → কারণ **NULL কখনো কারো সমান হয় না**, তাই ON শর্ত অজানা, সত্য নয়।

**মনে রাখার কৌশল:** শব্দটিকে **আগে-নাম দেওয়া** টেবিলের প্রতিশ্রুতি হিসেবে পড়ো। **LEFT**: বাম পাশের সব রাখো, ডান পাশ NULL দিয়ে পূরণ করো। **RIGHT**: শব্দটি বদলে দাও। **FULL OUTER**: দুই পাশের সব রাখো। **INNER**: শুধু যা মেলে। **CROSS**: কোনো প্রতিশ্রুতিই নেই।

## ৭ জয়েন প্যাটার্ন, সাব-কোয়েরি ও বাকি অধ্যায়

**প্যাটার্ন: কর্মচারী নেই এমন বিভাগসহ প্রতিটি বিভাগ:** ইঞ্জিনের ডিফল্ট জয়েন তাদের বাদ দেয়, তাই স্পষ্ট চাইতে হয় — `LEFT JOIN` দিয়ে — আর তারপর গোনতে হয় **যুক্ত টেবিলের** কলাম, * নয়।

```
SELECT d.dept_name, COUNT(e.emp_id) AS employee_count
FROM   Department d
LEFT JOIN Employee e ON e.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name
ORDER BY employee_count DESC;
```

**পরীক্ষার ফাঁদ ✍️ দুটি গণনা, আর বাম জয়েন বাতিল করা WHERE:** `COUNT(e.emp_id)` বাইরের জয়েনে তৈরি NULL বাদ দেয়, তাই খালি বিভাগের জন্য ঠিক **0** দেয়; `COUNT(*)` পূরণ করা সারিটি গুনে ভুলে **1** দেয়। আর `dept_name`-এর সঙ্গে `dept_id` দিয়েও গ্রুপ করলে একই নামের দুটি বিভাগ আলাদা থাকে। আর **ডান** টেবিলের শর্ত `WHERE`-এ দিলে সেটি জয়েনের *পরে* মূল্যায়িত হয়ে পূরণ করা সারিগুলো বাদ দেয়, আর আপনার LEFT JOIN নিঃশব্দে INNER JOIN হয়ে যায় — শর্তটি `ON`-এ দিন।

**সাব-কোয়েরির তিন প্রকার:** **স্কেলার** — তুলনার মধ্যে একটি মান: `WHERE salary > (SELECT AVG(salary) FROM Emp)`। **IN / EXISTS** — একটি সেট, কিংবা সত্য/মিথ্যা: `WHERE id IN (SELECT emp_id FROM Emp)`। **করেলেটেড** — বাইরের সারিকে উল্লেখ করে, তাই **প্রতি বাইরের সারির জন্য একবার** নতুন করে চলে: `WHERE e2.dept_id = e.dept_id`।

```
SELECT MAX(salary) AS second_highest        -- 2nd highest DISTINCT salary
FROM   Employee
WHERE  salary < (SELECT MAX(salary) FROM Employee);    -- NULL if there is none
-- and, above the average of the employee's OWN department:
SELECT e.name, e.salary, d.dept_name
FROM   Employee e JOIN Department d ON d.dept_id = e.dept_id
WHERE  e.salary > ( SELECT AVG(e2.salary) FROM Employee e2
                    WHERE e2.dept_id = e.dept_id );
```

উপরের করেলেটেড কোয়েরির **সমতুল্য** ডেরাইভড টেবিল রূপ: প্রতিটি বিভাগের গড় একবার হিসাব করে `JOIN (SELECT dept_id, AVG(salary) AS avg_sal FROM Employee GROUP BY dept_id) a` এ তারপর `WHERE e.salary > a.avg_sal` — সাধারণত এই পরিকল্পনাটিই দ্রুততর।

**পরীক্ষার ফাঁদ ✍️ তিনটি:** **১.** করেলেটেড সাব-কোয়েরি **প্রতি বাইরের সারির জন্য একবার** চলে, ফলে *e*-এর *নিজের* বিভাগের গড় পায়; প্রতিটি বিভাগের গড় আগে হিসাব করে নেওয়া ডেরাইভড টেবিল সাধারণত দ্রুততর পরিকল্পনা। **২.** দ্বিতীয়-সর্বোচ্চ কোয়েরিতে `<` ব্যবহার হয়, `≤` নয়, তাই সর্বোচ্চ বেতন একাধিকবার থাকলেও ঠিক থাকে, আর সব বেতন সমান হলে NULL দেয়। **৩.** সাব-কোয়েরিতে একটি NULL থাকলে `NOT IN (subquery)` **একটিও সারি ফেরত দেয় না** — মান অনুপস্থিত হতে পারে এমন জায়গায় `NOT EXISTS` ব্যবহার করো।

**এই অধ্যায়ের বাকি অংশ কোথায়:** **ফাংশনাল ডিপেন্ডেন্সি, 1NF–BCNF, নরমালাইজেশন** → **db3**। **ফাইল সংগঠন, প্রাইমারি ও সেকেন্ডারি ইনডেক্স, B+ ট্রি, কস্ট-বেসড অপটিমাইজেশন** — কোন ইনডেক্স কোন অ্যালজেব্রা-ট্রিকে সস্তা করে → **db4**। **ট্রানজ্যাকশন, ACID, আইসোলেশন, কনকারেন্সি, রিকভারি** → **db5**। **কনস্ট্রেইন্ট ও ট্রিগার** একই সিলেবাস লাইনে একটু নিচে, ডেটা মডেলের সাথেই দেখানো হয়েছে — **db1**। এখানে সেগুলো আবার বলা হয়নি।

# রাতের পরীক্ষার পাতা

**তথ্যপাতা:**

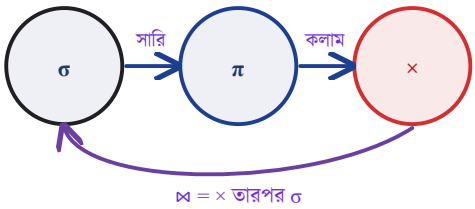
| | |
|---|---|
| σ = **সারি** (SQL-এর WHERE) | π = **কলাম** (SQL-এর SELECT তালিকা) |
| ρ = রিনেম · × = প্রতিটি সারি × প্রতিটি সারি | জয়েন = `σ(R × S)` |
| DDL: CREATE, ALTER, DROP, TRUNCATE | DML: SELECT, INSERT, UPDATE, DELETE |
| DCL: GRANT, REVOKE | TCL: COMMIT, ROLLBACK, SAVEPOINT |
| DROP কাঠামো ও ডেটা দুটোই মোছে | DELETE / TRUNCATE কাঠামো রাখে |
| চলার ক্রম: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY | WHERE সারি ছাঁকে **আগে** |
| HAVING গ্রুপ ছাঁকে **পরে** | COUNT(*) সারি গোনে, NULL-সহ |
| COUNT(col) শুধু non-NULL গোনে | ১০, NULL, ২০ এর AVG = **১৫** |
| বাকি অ্যাগ্রিগেট NULL বাদ দেয় | জয়েনে NULL কখনো কারো সমান হয় না |
| FULL OUTER = INNER + শুধু-LEFT + শুধু-RIGHT | ছয় জয়েন: INNER, LEFT, RIGHT, FULL, CROSS, SELF |
| CROSS JOIN = m × n সারি | কর্মচারী নেই এমন বিভাগ: `COUNT(e.emp_id)` |

## MCQ-র জন্য সেরা ১২টি এক-লাইনের কথা

1. সিলেকশন σ সারি ছাঁকে; প্রজেকশন π কলাম ছাঁকে
2. ρ নাম বদলায়; × প্রতিটি সারিকে প্রতিটি সারির সঙ্গে জোড়া করে
3. জয়েন = প্রোডাক্ট + সিলেকশন
4. DDL · DML · DCL · TCL — চার পরিবার, চার কাজ
5. **DROP** কাঠামো ও ডেটা একসাথে মোছে
6. **DELETE** ও **TRUNCATE** দুটোই কাঠামো রাখে
7. গ্রুপিংয়ের আগে WHERE, পরে HAVING
8. অ্যাগ্রিগেট ব্যবহার করতে পারে শুধু HAVING
9. ১, NULL, ৩, NULL, ৫ তে COUNT(*)=5, COUNT(a)=3
10. ১০, NULL, ২০ এর AVG = ১৫, কখনো ১০ নয়
11. জয়েনের শর্তে NULL কখনো কারো সমান হয় না
12. কর্মচারী নেই এমন বিভাগে `COUNT(e.emp_id)`

**সবচেয়ে বড় ফাঁদগুলো:** σ আর π অদল-বদল · π নকল সরায় কিন্তু SELECT সরায় না · DROP বনাম DELETE বনাম TRUNCATE · WHERE ও HAVING-এর গুলিয়ে ফেলা · AVG-এ NULL-কে 0 ধরা · NULL বিভাগের কর্মচারীকে inner join-এ খোঁজা · কর্মচারী নেই এমন বিভাগের জন্য `COUNT(*)` · ডান টেবিলের WHERE দিয়ে LEFT JOIN-কে INNER বানানো।

**সব কৌশল একসাথে:** সিগমা সারি বাছে, পাই কলাম বাছে · আগে প্রোডাক্ট, তারপর বাছাই · যেখানে আগে, হেভিং পরে · তারকা প্রতিটি সারি দেখে, কলাম শুধু আসল উত্তর দেখে · বাম পাশের সব রাখো, ডান পাশ NULL দিয়ে পূরণ করো · জয়েন করো অ্যালিয়াস দিয়ে, খালি টেবিল নাম দিয়ে নয়।




দুটি মৌলিক অপারেশনেই সব জয়েন

## ✓ অনুশীলনী MCQ (পরীক্ষার ধরনে, ১০ নম্বর)

**১.** কোন SQL কমান্ড একটি টেবিলের কাঠামো ও তার সব ডেটা একসঙ্গে মুছে ফেলে?
ক) DELETE খ) TRUNCATE
গ) ALTER ঘ) DROP

**২.** `GROUP BY` প্রয়োগের পর গ্রুপগুলো ছাঁকে কোন ক্লজ?
ক) HAVING খ) WHERE
গ) ORDER BY ঘ) DISTINCT

**৩.** কোন relational algebra operator প্রদত্ত শর্ত পূরণকারী টিপলগুলো রাখে?
ক) Projection (π) খ) Rename (ρ)
গ) Selection (σ) ঘ) Cartesian product (×)

**৪.** `T`-তে একটি কলাম `a` এবং সারি 1, NULL, 3, NULL, 5। `SELECT COUNT(*), COUNT(a) FROM T;` দেয়:
ক) 3 এবং 3 খ) 5 এবং 3
গ) 5 এবং 5 ঘ) 3 এবং 5

**৫.** `R.score`-এ আছে 10, NULL, 20। `SELECT AVG(score) FROM R;` ফেরত দেয়:
ক) 10 খ) 15
গ) 20 ঘ) NULL

**৬.** `Emp` = (1,10),(2,20),(3,NULL),(4,10) এবং `Dept` = (10,'IT'),(20,'HR'),(30,'Finance') হলে `dept_id`-তে INNER JOIN কতটি সারি ফেরত দেয়?
ক) 3 সারি খ) 4 সারি
গ) 5 সারি ঘ) 12 সারি

**৭.** একই ডেটাতে FULL OUTER JOIN কতটি সারি ফেরত দেয়?
ক) 3 সারি খ) 4 সারি
গ) 5 সারি ঘ) 6 সারি

**৮.** `Sales` = ('N',100),('N',200),('S',50),('S',70),('E',300)। `GROUP BY region HAVING SUM(amount) > 150` ফেরত দেয়:
ক) 1 সারি খ) 3 সারি
গ) 5 সারি ঘ) 2 সারি

**৯.** খালি বিভাগসহ প্রতিটি বিভাগের কর্মচারী-সংখ্যা দেখাতে কী লাগে?
ক) COUNT(*) খ) COUNT(e.emp_id)
গ) COUNT(d.dept_name) ঘ) SUM(e.salary)

**১০.** SQL-এ relational algebra-র selection σ কোনটির সঙ্গে মেলে?
ক) WHERE ক্লজ খ) SELECT তালিকা
গ) GROUP BY ঘ) ORDER BY

উত্তরমালা → ১-ঘ · ২-ক · ৩-গ · ৪-খ · ৫-খ · ৬-ক · ৭-গ · ৮-ঘ · ৯-খ · ১০-ক

**লিখিত প্রশ্ন:** `Students(id, name, dept)` ও `Courses(cid, name)`-এর উপর রিলেশনাল অ্যালজেব্রার মৌলিক অপারেশনগুলো ব্যাখ্যা করো। DDL, DML, DCL ও TCL আলাদা করো এবং বলো DROP, DELETE, TRUNCATE, ALTER প্রতিটি ঠিক কী করে। তারপর `Emp` = (1,10),(2,20),(3,NULL),(4,10) ও `Dept` = (10,'IT'),(20,'HR'),(30,'Finance')-এর জন্য INNER, LEFT, RIGHT, FULL OUTER ও CROSS JOIN-এর সারির সংখ্যা দাও এবং ব্যাখ্যা করো কর্মচারী ৩ কেন inner join-এ কখনো আসে না। ১০ নম্বর

১ **RA-র অপারেটর (২)** — **σ** শর্ত পূরণ করা সারি রাখে, **π** কলাম রাখে ও নকল সরায়, **ρ** নাম বদলায়, × এক রিলেশনের প্রতিটি সারিকে অন্যের প্রতিটি সারির সঙ্গে জোড়া করে (২ × ২ = **৪**), ∪ − ∩ সেট অপারেটর, আর ⋈ তৈরি হয় σ(R × S) থেকে। ২

২ **কমান্ড পরিবার (২)** — **DDL** (Definition) কাঠামোর উপর: CREATE, ALTER, DROP, TRUNCATE। **DML** (Manipulation) সারির উপর: SELECT, INSERT, UPDATE, DELETE। **DCL** (Control) অধিকারের উপর: GRANT, REVOKE। **TCL**: COMMIT, ROLLBACK, SAVEPOINT। ২

৩ **চারটি কমান্ড (২)** — **DROP** কাঠামো ও ডেটা দুটোই সরায়; **DELETE** সারি সরায়, চাইলে `WHERE` দিয়ে, কাঠামো রাখে; **TRUNCATE** সব সারি সরায়, কাঠামো রাখে, `WHERE` নেয় না; **ALTER** কাঠামো বদলায়, ডেটা রাখে। ২

৪ **সারির সংখ্যা (১)** — INNER **3**, LEFT **4**, RIGHT **4**, FULL OUTER **5** = 3 + 1 + 1, CROSS 4 × 3 = **12**। 1

৫ **কর্মচারী ৩ কেন অদৃশ্য (২)** — তার `dept_id` NULL আর **NULL = 10 অজানা, সত্য নয়**; inner join কেবল তখনই সারি রাখে যখন ON শর্ত **সত্য**। ২

৬ **WHERE বনাম HAVING (১)** — `WHERE` গ্রুপিংয়ের **আগে সারি** ছাঁকে; `HAVING GROUP BY`-এর **পরে গ্রুপ** ছাঁকে, আর অ্যাগ্রিগেট ব্যবহার করতে পারে একমাত্র এটি। ১

### পরীক্ষার আগে ✓

- ✓ σ, π, ρ, × বলতে পারি, প্রতিটির উদাহরণসহ
- ✓ জয়েন σ(R × S) হিসেবে লিখতে পারি, ρ ব্যাখ্যা করতে পারি
- ✓ যেকোনো কমান্ড DDL, DML, DCL বা TCL-এ বসাতে পারি
- ✓ DROP, DELETE, TRUNCATE, ALTER কী করে বলতে পারি
- ✓ ছয় জয়েনের সারির সংখ্যা গোনতে পারি
- ✓ COUNT(*) ও AVG-এর NULL নিয়ম বলতে পারি